

METU Computer Engineering

CEng 111 – Fall 2026
Week 6b

Credit: Some slides are from the “Invitation to Computer Science” book by G. M. Schneider, J. L. Gersting and some from the “Digital Design” book by M. M. Mano and M. D. Ciletti.

Previously on CENG111!

ASCII
7 bits long

Dec	Bin	Char	Dec	Bin	Char	Dec	Bin	Char	Dec	Bin	Char
0	0000 0000	[NUL]	32	0010 0000	space	64	0100 0000	@	96	0110 0000	`
1	0000 0001	[SOH]	33	0010 0001	!	65	0100 0001	A	97	0110 0001	a
2	0000 0010	[STX]	34	0010 0010	"	66	0100 0010	B	98	0110 0010	b
3	0000 0011	[ETX]	35	0010 0011	#	67	0100 0011	C	99	0110 0011	c
4	0000 0100	[EOT]	36	0010 0100	$	68	0100 0100	D	100	0110 0100	d
5	0000 0101	[ENQ]	37	0010 0101	%	69	0100 0101	E	101	0110 0101	e
6	0000 0110	[ACK]	38	0010 0110	&	70	0100 0110	F	102	0110 0110	f
7	0000 0111	[BEL]	39	0010 0111	'	71	0100 0111	G	103	0110 0111	g
8	0000 1000	[BS]	40	0010 1000	(	72	0100 1000	H	104	0110 1000	h
9	0000 1001	[TAB]	41	0010 1001	)	73	0100 1001	I	105	0110 1001	i
10	0000 1010	[LF]	42	0010 1010	*	74	0100 1010	J	106	0110 1010	j
11	0000 1011	[VT]	43	0010 1011	+	75	0100 1011	K	107	0110 1011	k
12	0000 1100	[FF]	44	0010 1100	,	76	0100 1100	L	108	0110 1100	l
13	0000 1101	[CR]	45	0010 1101	-	77	0100 1101	M	109	0110 1101	m
14	0000 1110	[SO]	46	0010 1110	.	78	0100 1110	N	110	0110 1110	n
15	0000 1111	[SI]	47	0010 1111	/	79	0100 1111	O	111	0110 1111	o
16	0001 0000	[DLE]	48	0011 0000	0	80	0101 0000	P	112	0111 0000	p
17	0001 0001	[DC1]	49	0011 0001	1	81	0101 0001	Q	113	0111 0001	q
18	0001 0010	[DC2]	50	0011 0010	2	82	0101 0010	R	114	0111 0010	r
19	0001 0011	[DC3]	51	0011 0011	3	83	0101 0011	S	115	0111 0011	s
20	0001 0100	[DC4]	52	0011 0100	4	84	0101 0100	T	116	0111 0100	t
21	0001 0101	[NAK]	53	0011 0101	5	85	0101 0101	U	117	0111 0101	u
22	0001 0110	[SYN]	54	0011 0110	6	86	0101 0110	V	118	0111 0110	v
23	0001 0111	[ETB]	55	0011 0111	7	87	0101 0111	W	119	0111 0111	w
24	0001 1000	[CAN]	56	0011 1000	8	88	0101 1000	X	120	0111 1000	x
25	0001 1001	[EM]	57	0011 1001	9	89	0101 1001	Y	121	0111 1001	y
26	0001 1010	[SUB]	58	0011 1010	:	90	0101 1010	Z	122	0111 1010	z
27	0001 1011	[ESC]	59	0011 1011	;	91	0101 1011	[	123	0111 1011	{
28	0001 1100	[FS]	60	0011 1100	<	92	0101 1100	\	124	0111 1100	|
29	0001 1101	[GS]	61	0011 1101	=	93	0101 1101	]	125	0111 1101	}
30	0001 1110	[RS]	62	0011 1110	>	94	0101 1110	^	126	0111 1110	~
31	0001 1111	[US]	63	0011 1111	?	95	0101 1111	_	127	0111 1111	[DEL]



METU Computer Engineering

Previously on CENG111!

How about a text?

- Text in a computer has two alternative representations:

1. A fixed-length number representing the length of the text followed by the binary values of the characters in the text.
   - Ex: “ABC” =>

   00000011 01000001 01000010 01000011 (3 ‘A’ ‘B’ ‘C’)

2. Binary values of the characters in the text ended with a unique marker, like “00000000” which has no value in the ASCII table.
   - Ex: “ABC” =>

   01000001 01000010 01000011 00000000 (‘A’ ‘B’ ‘C’ END)

The Reliability of Binary Representation

Previously on CENG111!

- Electronic devices are most reliable in a bistable environment
- Bistable environment
  - Distinguishing only two electronic states
    - Current flowing or not, or
    - Direction of flow
- Computers are bistable: hence binary representations
- See the following video for contrasting the design of a binary computer and a ternary one:

  https://www.youtube.com/watch?v=fXwSFhUVFmE



Previously on CENG111!

What is data?

- CPU can only understand two types of data:
  - Integers,
  - Floating points.
- The following are not directly understandable by a CPU:
  - Characters (‘a’, ‘A’, ‘2’, …)
  - Strings (“apple”, “banana”, …)
  - Complex Numbers
  - Matrices
  - Vectors
- But, programming languages can implement these data types.



Previously on CENG111!

Problems due to precision loss

- What has precision loss is misleading: 0.9 has precision loss whereas 0.9375 does not!
  - 0.9375 = 0.5 + 0.25 + 0.125 + 0.0625
  - Therefore: $(0.9375)_{10} = (0.1111)_2$
- Imprecise representations are round-offs!
- If you have a lot of round-offs, you might be in trouble!



Previously on CENG111!

Integers in Python

- Python provides `int` type.

```
>>> type(3)
<type 'int'>
>>> type(3+4)
<type 'int'>
>>>
```

- For big integers, Python v2 has the `long` type:

```
>>> type(3L)
<type 'long'>
>>> type(3L+4L)
<type 'long'>
>>>
```

`int` is limited by the hardware whereas `long` type is unlimited in Python.

Previously on CENG111!

Floating Points in Python

- Python provides `float` type.

```
>>> type(3.4)
<type 'float'>
>>>
```

```
>>> 3.4+4.3
7.7
>>> 3.4 / 4.3
0.7906976744186046
```

See the following document for how Python handles precision loss:
https://docs.python.org/3/tutorial/floatingpoint.html

Previously on CENG111!

Characters in Python

- Python does not have a separate data type for characters!
- However, one-character strings can be treated like characters:
  - **ord**(One_Char_String) : returns the ASCII value of the character in One_Char_String.
    - Ex: ord("A") returns 65.
  - **chr**(ASCII_value) : returns the character in a string that has the ASCII value ASCII_value:
    - Ex: chr(66) returns "B"



Previously on CENG111!

Boolean Values in Python

- Python provides the `bool` data type for boolean values.
- `bool` data type can take `True` or `False` as values.

```
>>> 3 > 4
False
>>> type(3 > 4)
<type 'bool'>
```

not operator:

```
not True
not 4 > 3
```

```
>>> True == 2
False
>>> True == 1
True
>>> False == 0
True
>>> False == 1
False
```

Try this:

```
>>> True == bool(2)
```



METU Computer Engineering

Today

- Structured (container) data types

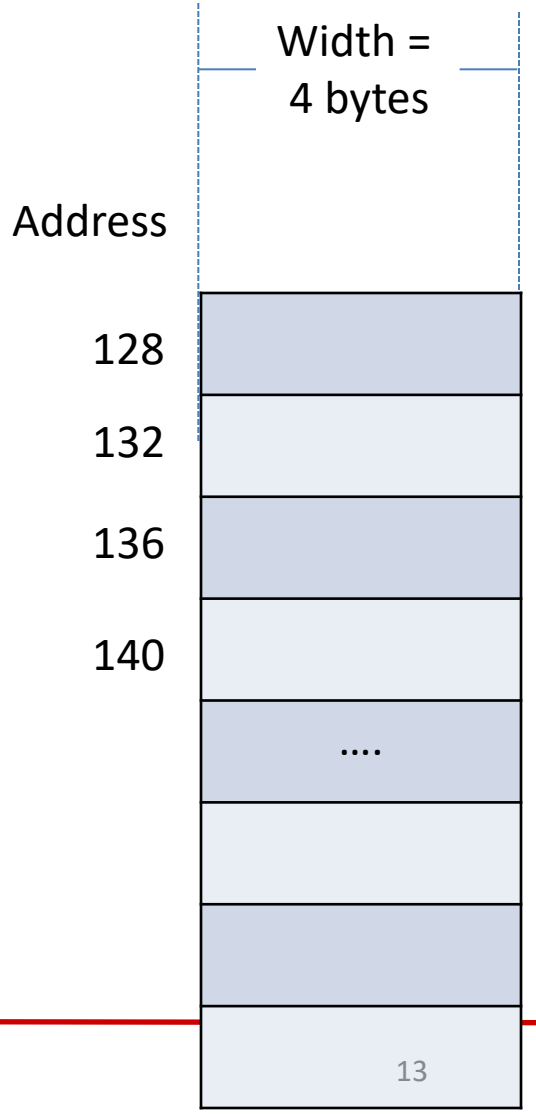


Administrative Notes

- Midterm Exam
  - Date: 20 November, 18:00
- Lab Exams
  - Lab Exam 1: 8 November, Sunday, 13:00
  - Lab Exam 2: 6 December, Sunday, 13:00
  - Lab Exam 3: 20 December, Sunday, 13:00
  - Lab Exam 4: 3 January, Sunday, 13:00

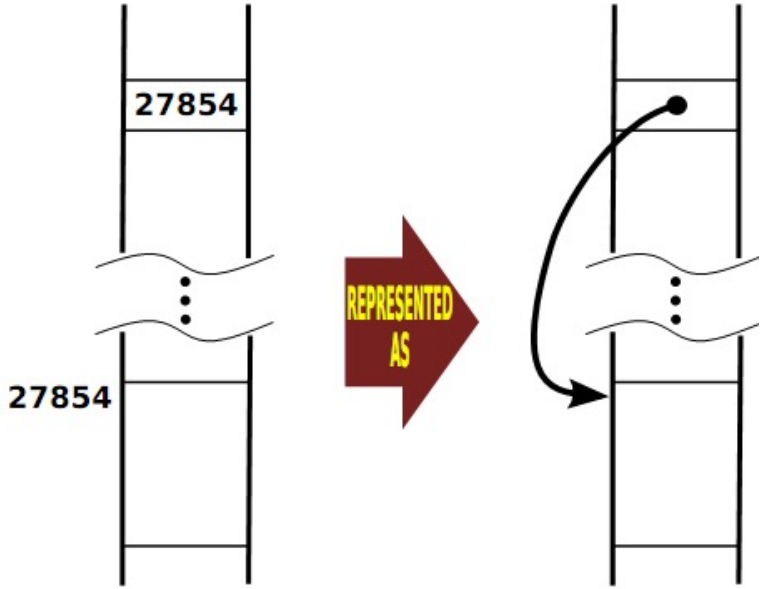
Structured Data

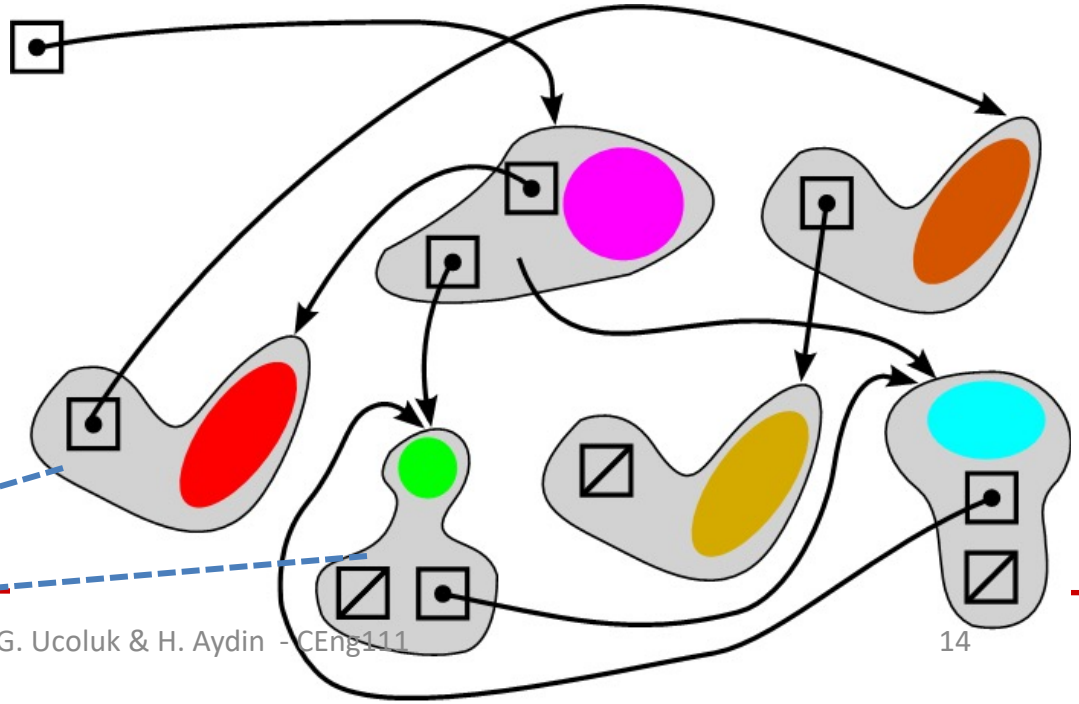
- If you have lots and lots of one type of data (for example, the ages of all the people in Turkey):
  - You can store them into memory consecutively (supported by most PLs)
    - This is called *arrays*.
  - Easy to access an element. Nth element:
    - <Starting-address>+ (N-1)*<Word Width>
    - Ex: 2nd element is at 128 + (2-1) * 4 = 132



Structured Data

- What if you have to make a lot of deletions and insertions in the middle of an array?
- Then, you have to store your data in blocks/units such that each unit has the starting address of the next unit/block.





Blocks of Data

METU Computer Engineering



Strings

- Sequence of characters:
  - Ex: “Book”, “Programming”, “Python”
- How can they be represented?
  1. Put a set of characters one after the other and end them with a non-character value.
  2. At the beginning of the characters, specify how many characters follow.
- Both have advantages and disadvantages.



Strings in Python

Python provides the **str** data type for strings:

```
>>> "Hello?"
'Hello?'
>>> type("Hello?")
<type 'str'>
```

- Simplest operation with a string:

```
>>> len("Hello?")
6
```

Strings in Python

- Accessing elements of a string
  "Hello?"[0] → 1st character (i.e., "H")
  "Hello?"[4] → 5th character
- Indexing starts at 0!!!
- What is the last element then?
  - "Hello?"[len("Hello?") - 1]
- Negative indexing possible:
  - Last element: "Hello?"[-1] → "?"
- In general:
  - String[start:end:step]
  - Ex: "Hello?"[0:4:2] → "Hl"
  - Ex: "Hello?"[2:4] → "ll"

Creating Strings in Python

1. Enclosing a set of characters between quotes:
   - “ali”, “veli”, “deli”, …
2. Using the str() function:
   - str(4.5) → “4.5”
3. Using the raw_input() function:

```
>>> a = raw_input("--> ")
--> Do as I say
>>> a
'Do as I say'
>>> type(a)
<type 'str'>
```

Python v2

```
>>> a = input("--> ")
--> Do as I say
>>> a
'Do as I say'
>>> type(a)
<class 'str'>
>>>
```

Python v3



Internal of Python's String Implementation

(taken from https://www.laurentluce.com/posts/python-string-objects-implementation/)

- PyStringObject structure
  - "A string object in Python is represented internally by the structure PyStringObject. "ob_shash" is the hash of the string if calculated. "ob_sval" contains the string of size "ob_size". The string is null terminated. The initial size of "ob_sval" is 1 byte and ob_sval[0] = 0. If you are wondering where "ob_size is defined", take a look at PyObject_VAR_HEAD in object.h."

```
typedef struct {
    PyObject_VAR_HEAD
    long ob_shash;
    int ob_sstate;
    char ob_sval[1];
} PyStringObject;
```

METU Computer Engineering



Tuples

- Tuple: ordered set of data:
  - (1, 2, 3)
  - (“a”, “b”, “c”)
- May be heterogeneous:
  - (“Salary”, 2000, “Age”, 25, “Birth”, “Ankara”)



Tuples in Python

```
>>> (1, 2, 3, 4, "a")
(1, 2, 3, 4, 'a')
>>> type((1, 2, 3, 4, "a"))
<type 'tuple'>
```

- Tuples in Python: collection of data enclosed in parentheses, separated by comma.
- Accessing elements of a tuple (like strings):
  - Positive Indexing: `(1, 2, 3, 4, "a")[2]` returns 3.
  - Negative Indexing: `(1, 2, 3, 4, "a")[-1]` returns `'a'`.
  - Ranged Indexing, *i.e.*, `[start:end:step]`: `(1, 2, 3, 4, "a")[0:4:2]` leads to `(1, 3)`.



Creating Tuples in Python

METU Computer Engineering

1. Enclosing data within parentheses:
   - Ex: (1, “a”, “cde”, 23)
2. Using the tuple() function:
   - Ex: tuple(“ABC”) → (‘A’, ‘B’, ‘C’)
3. Using the input() function:

```
>>> a = input("Give me a tuple:")
Give me a tuple:(1, 2, 3)
>>> a
(1, 2, 3)
>>> type(a)
<type 'tuple'>
```

Python v2

```
>>> a = eval(input("Give me a tuple:"))
Give me a tuple:(1,2,3)
>>> a
(1, 2, 3)
>>> type(a)
<class 'tuple'>
```

Python v3



Lists

- Similar to tuples.
- Difference:
  - Tuples are immutable (i.e., not changeable) whereas lists are mutable.

Lists in Python

```
>>> [1, 2, 3, 4, "a"]
[1, 2, 3, 4, 'a']
>>> type([1, 2, 3, 4, "a"])
<type 'list'>
```

- Lists in Python: collection of data enclosed in brackets, separated by comma.
- Accessing elements of a list (like strings & tuples):

- Positive Indexing: `[1, 2, 3, 4, "a"][2]` returns 3.
- Negative Indexing: `[1, 2, 3, 4, "a"][-1]` returns `'a'`.
- Ranged Indexing, *i.e.*, `[start:end:step]`: `[1, 2, 3, 4, "a"][0:4:2]` leads to `[1, 3]`.

Creating Lists in Python

METU Computer Engineering

1. Enclosing data within brackets:
   - Ex: [1, “a”, “cde”, 23]
2. Using the list() function:
   - Ex: list(“ABC”) → [‘A’, ‘B’, ‘C’]
3. Using the range() function:

```
range(    [start,] stop[, step])
```

   - Ex: range(1, 10, 2) → [1, 3, 5, 7, 9]
4. Using the input() function:

```
>>> a = input("Give me a list:")
Give me a list:[1, 2, "a"]
>>> a
[1, 2, 'a']
>>> type(a)
<type 'list'>
```

Python v2

```
>>> a = eval(input("Give me a list:"))
Give me a list:[1,2,"a"]
>>> a
[1, 2, 'a']
>>> type(a)
<class 'list'>
```

Python v3



Modifying a List in Python

- List[range] = Data
  - Ex:

```
>>> L = [3, 4, 5, 6, 7, '8', 9, '10']
>>> L[::2]
[3, 5, 7, 9]
>>> L[::2] = [4, 6, 8, 10]
>>> L[::2]
[4, 6, 8, 10]
>>> L[]
[4, 4, 6, 6, 8, '8', 10, '10']
```

- Using the append() function:
  - List.append(item)
  - Ex: [1, 2, 3].append(5) → [1, 2, 3, 5]



Modifying a List in Python

- Using the extend() function:
  - List.extend(Another_list)
  - Ex: [1, 2, 3].extend([“a”, “b”]) → [1, 2, 3, “a”, “b”]

```
>>> L.extend(["a", "b"])
>>> L
[4, 4, 6, 6, 8, '8', 10, '10', 'a', 'a', 'b']
```

- Using the insert() function:
  - List.insert(index, item)

```
>>> L=[1, 2, 3]
>>> L
[1, 2, 3]
>>> L.insert(1, 0)
>>> L
[1, 0, 2, 3]
```

Removing Elements from a List in Python

- **del statement:** `del L[start:end]`

```
>>> L
[1, 0, 2, 3]
>>> del L[1]
>>> L
[1, 2, 3]
```

- **L.pop() function:** `L.pop([index])`

```
>>> L=[1,2,3]
>>> L.pop()
3
>>> L
[1, 2]
>>> L.pop(0)
1
>>> L
[2]
```

- **L.remove() function:** `L.remove(value)`

```
>>> L
[2, 1, 3]
>>> L.remove(1)
>>> L
[2, 3]
```



A frequent operation with containers

- Membership
  - in
  - not in
- “en” in “deneme”
  - True
- “an” in “deneme”
  - False
- “dem” in “deneme”
  - False

METU Computer Engineering